

Building a Chatbot_ Context, Memory, Streaming & Hallucinations

Coder Army

Modern AI applications such as ChatGPT feel like they are having one continuous conversation with us.

But underneath, an LLM works very differently.

To build a good chatbot, we need to understand the difference between:

- an **LLM**
 - a **conversation**
 - **conversation history**
 - a **context window**
 - **application memory**
 - **model training**
-

1. A Model Call Is Not Automatically a Conversation

A raw LLM does not automatically remember previous independent API calls.

Consider a conversation:

```
User: My food hasn't arrived.  
  
Assistant: I am sorry for the trouble.  
Would you like to wait or request a refund?  
  
User: I want a refund.
```

From the user's point of view, this feels like one continuous conversation.

But conceptually, the model may simply receive:

```
Request 1  
→ Response 1  
  
Request 2  
→ Response 2  
  
Request 3  
→ Response 3
```

Each API call can be independent.

The **application** is responsible for creating the conversational experience.

The LLM generates. The application creates the experience around that generation.

2. Maintaining Conversation History

Suppose our chatbot receives:

```
My food hasn't arrived.
```

The model responds:

```
I am sorry for the trouble.  
Would you like to wait or request a refund?
```

Now the user says:

```
Yes.
```

If we send only `"Yes"` in the next API request, the model may not know what the user is agreeing to.

Instead, we provide the relevant conversation again:

USER:
My food hasn't arrived.

ASSISTANT:
I am sorry for the trouble.
Would you like to wait or request a refund?

USER:
I want a refund.

The application maintains the history and sends the relevant information to the model.

3. Messages and Roles

A **role** tells the model what kind of message it is receiving.

The two most common conversational roles are:

User
Assistant

User Role

The **User** role represents something the user is asking or providing.

Example:

USER:

I ordered my food three hours ago.
The tracking page hasn't updated for an hour.

This is the actual request or information coming from the user.

Assistant Role

The **Assistant** role represents something previously generated by the assistant.

Example:

ASSISTANT:

Your order appears delayed.
Would you like to wait another 10 minutes
or proceed with cancellation?

Why should we send previous assistant messages back to the model?

Because future user messages may depend on them.

For example:

USER:

Yes, I can wait, but if its quality is affected,
I want my money back.

The meaning of words such as **"its"** depends on the previous conversation.

Therefore, both user and assistant messages may be necessary context.

4. The Problem With Putting Everything in the User Prompt

Suppose we want our chatbot to:

- remain professional
- answer only food-ordering questions
- avoid speculation
- focus on customer-support issues

We could append these instructions to every user message.

But then the input starts mixing two different things:

Application Instructions
+

These should ideally have different semantic roles.

This is where the **System role** becomes useful.

5. System Role

System-level instructions describe how the assistant should behave.

Example:

SYSTEM:

You are a customer-support executive for Tomato,
a food-ordering application.

Answer questions related to food orders,
tracking, payments, refunds and company policies.

Be professional and empathetic.

Do not invent missing information.

If the user asks something unrelated to food ordering,
politely explain that you cannot help with that request.

Think of the system message as:

The application's standing instructions for the model.

6. Why System Instructions Matter

System instructions and user messages serve different purposes.

System instructions

Control the expected behaviour of the application.

User instructions

Represent the user's current request.

Conceptually:

```
Application Rules
  ↓
User Request
```

The user interacts with the assistant within the behaviour defined by the application.

For example:

```
SYSTEM:

You are a programming tutor.
Guide students using hints instead of giving
complete solutions immediately.

USER:

Give me the complete solution.
```

The application is attempting to preserve its tutoring behaviour even when the user asks for something different.

7. System Prompts Do Not Retrain the Model

Suppose we provide:

```
You are a customer-support assistant.
```

The LLM has not suddenly become a permanently trained customer-support model.

We did not change its neural-network weights.

Instead:

Original Trained Model
+
Current Instructions
↓
Behaviour for this request

The system prompt becomes part of the context influencing generation.

8. What Can a System Instruction Define?

A useful system instruction can specify:

1. ROLE
2. TASK
3. BEHAVIOUR
4. CONSTRAINTS

Example:

ROLE:
You are a customer-support executive for Tomato.

TASK:
Help customers with food-ordering problems.

BEHAVIOUR:
Use professional and empathetic language.

CONSTRAINTS:
Do not answer unrelated questions.
Do not invent order details.

This makes application behaviour much easier to manage.

9. System Instructions Are Not a Complete Security Mechanism

System instructions are powerful, but an LLM is still probabilistic.

A production application should not depend only on prompts.

Reliable systems can also use:

- application logic
- permissions
- input/output validation
- structured outputs
- tools
- guardrails
- external verification

Prompts influence behaviour, but application engineering is still required.

10. The Illusion of Memory

Suppose yesterday the user said:

I teach Java.

Today the user asks:

What do I teach?

and the application answers:

Java.

It feels like:

The LLM remembered me.

But the application may actually be doing:

```
Memory Store / Database
    ↓
Retrieve User Information
    ↓
Add Information to Current Context
    ↓
LLM
```

The memory may belong to the **application**, not permanently to the model itself.

11. Context Window

Now imagine our conversation contains:

```
SYSTEM:
You are a programming tutor.

USER:
Explain Docker.

ASSISTANT:
Docker lets us package an application...

USER:
Explain it with an analogy.
```

Can we keep adding messages forever?

No.

An LLM can process only a limited amount of information for a request.

That limited working area is called the:

| Context Window

12. Understanding the Context Window Intuitively

Imagine somebody gives you a sheet of paper containing:

```
Instructions
Previous conversation
Current question
Documents
Examples
```

Then says:

| Answer the final question using the information available on this sheet.

You can directly work with whatever fits on that sheet.

For an LLM, that metaphorical sheet is its **context window**.

The size is generally measured in:

```
TOKENS
```

So conceptually:

```
Context Window
=
Amount of tokenized information
the model can process for an interaction
```

13. What Can Enter the Context?

A model request may contain:

```
System Instructions
+
Previous User Messages
```

+
Previous Assistant Messages
+
Current User Message
+
Retrieved Information
+
Tool Results

All of this can influence the next generation.

This is why the model does not simply respond to the user's last sentence.

It responds based on the **context available to it**.

14. Context Is Different From Model Knowledge

Two concepts must remain separate:

Model Parameters
vs
Context

Model Parameters

These are learned during training.

Conceptually:

Training
↓
Model Weights
↓
Learned Patterns and Capabilities

Context

Context is information supplied during the current interaction.

```
Current Information
  ↓
Context
  ↓
Influences Generation
```

The context can change from one request to another without retraining the model.

15. Input and Output Both Need Token Capacity

The system also needs room for generated output.

Conceptually:

```
Input Context
+
Generated Output
```

both use the model's finite token-processing capacity.

Therefore, if an application fills almost all available capacity with input, it cannot expect unlimited output afterward.

16. Conversation History vs Context Window

These terms are related but not identical.

Conversation History

The complete sequence of exchanges stored by the application:

User → Assistant → User → Assistant → ...

The history might live in:

RAM
Database
Redis
Browser State
Application State

Context Window

The information actually supplied to the model for the **current generation**.

For example:

Stored Conversation History:
1000 messages

Information sent to the model:
Last 20 relevant messages
+
System instructions
+
Current request

So an application can store much more history than it sends to the model.

17. Why Chat Requests Become Larger Over Time

Imagine a conversation grows like this:

Turn 1:
[A]

Turn 2:

[A B]

Turn 3:

[A B C]

Turn 4:

[A B C D]

If previous messages are repeatedly included, later requests contain more input tokens.

Therefore:

A longer conversation can mean more input tokens per request.

18. Simple Token Example

Suppose:

System Prompt = 100 tokens
Each Conversation Turn $\approx$ 200 tokens
Current Question = 50 tokens

Early conversation:

$100 + 200 + 50$
= 350 input tokens

After 20 relevant turns:

100
+
(20 $\times$ 200)
+
50

= 4,150 input tokens

The newest user message may still contain only 50 tokens.

But the actual API request can be much larger because previous context is also being included.

19. Context Management

Production applications usually should not keep sending everything forever.

Instead, they can decide which information is useful.

Possible strategies include:

Last N messages

or:

First few messages
+
Most recent messages

or:

Summary of older conversation
+
Recent conversation

or:

Only relevant information

Conceptually:

Full Conversation History
↓
Context Management
↓
Relevant Context

LLM

This becomes an important part of building good GenAI applications.

20. Bigger Context Window Does Not Mean "Send Everything"

Suppose a model supports a very large context window.

That does not mean we should blindly fill it.

Imagine asking an employee:

| What did Rohit say about Java?

Instead of providing the relevant paragraph, you give them hundreds of thousands of pages of unrelated information.

That is not necessarily helpful.

More information does not automatically mean better information.

A good AI application should focus on:

Relevant Context

instead of simply:

Maximum Context

| **The engineering challenge is not merely giving the model more context. It is giving it the right context.**

This is one of the most important ideas for a Forward Deployed Engineer.

21. Same Question, Different Context

Consider the exact same question:

Explain Docker.

Now change only the surrounding context.

Context A — Complete Beginner

You are a System Architect.

The user is a complete beginner.

Avoid technical jargon.

Use a simple real-world analogy.

User:

Explain Docker.

The answer might begin with an analogy involving shipping containers.

Context B — Experienced Backend Engineer

You are a System Architect.

The user is an experienced backend engineer
familiar with Linux, process isolation
and deployment.

Explain Docker technically and concisely.

User:

Explain Docker.

Now the response may discuss:

```
OS-level isolation
containers
images
namespaces
deployment
```

The user question did not change.

The model was not retrained.

What changed?

```
CONTEXT
```

22. Next-Token Prediction Depends on Context

At a high level:

```
P(next token | context)
```

In simple words:

What is the likely next token given everything the model has received so far?

If the context changes:

```
Context A
  ↓
P(next token | Context A)
```

will be different from:

Context B

↓

P(next token | Context B)

Therefore, different context can produce different generations.

The model does not respond to the final user sentence in isolation. It generates based on the context available to it.

23. Does an LLM Learn From Our Prompt?

Consider:

USER:

Whenever I say "Army", I mean "Coder Army".

USER:

What does Army mean?

ASSISTANT:

Coder Army.

It can feel like:

I taught the AI something.

But this is very different from model training.

24. What Learning Means During Model Training

During training, conceptually:

Training Data

↓

Model Makes Predictions
↓
Prediction Compared With Expected Result
↓
Error Calculated
↓
Model Parameters Adjusted
↓
Repeat

Eventually:

Trained Model
=
Architecture
+
Learned Parameters

In the technical machine-learning sense, learning generally involves changing model parameters based on data.

25. What Happens During a Normal Prompt?

Suppose we tell the model:

My company's internal project is called GateKeeper.

Then ask:

What is the name of my project?

If the previous information is available in context, the model can answer:

GateKeeper.

Conceptually:

```
Prompt / Context
  ↓
Existing Model Weights
  ↓
Response
```

What did not happen simply because of this normal request:

```
Prompt
  ↓
Backpropagation
  ↓
Modify Model Parameters
  ↓
Create Newly Trained Model
```

Therefore:

Using information from context is not the same as training the model on that information.

26. Application Memory vs Model Training

If an AI application remembers something across conversations, it may be doing:

```
Memory Store
  ↓
Retrieve User Information
  ↓
Add to Current Context
  ↓
LLM
```

Therefore:

```
Application Memory
≠
```

This distinction becomes very important when building personalised AI applications.

27. Streaming

Once we understand next-token generation, streaming becomes easy to understand.

Suppose the answer is:

```
Docker is a containerization platform.
```

The model generates progressively.

Conceptually:

```
Docker
```

then:

```
Docker is
```

then:

```
Docker is a
```

then:

```
Docker is a containerization
```

and so on.

If tokens are already being generated gradually, the application does not always need to wait for the full response before displaying anything.

That is the basic idea behind:

Streaming

28. Without Streaming

Traditional request-response flow:

```
Client
  ↓
Request
  ↓
Server / LLM

Generate...
Generate...
Generate...

  ↓
Complete Response
  ↓
Client
```

The user may see:

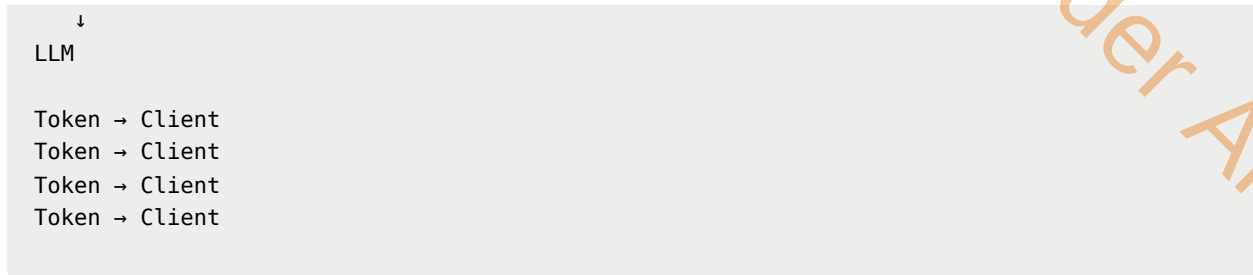
```
Loading...
```

until the complete answer arrives.

29. With Streaming

With streaming:

```
Client
  ↓
Request
```



The user starts seeing the response while generation is still happening.

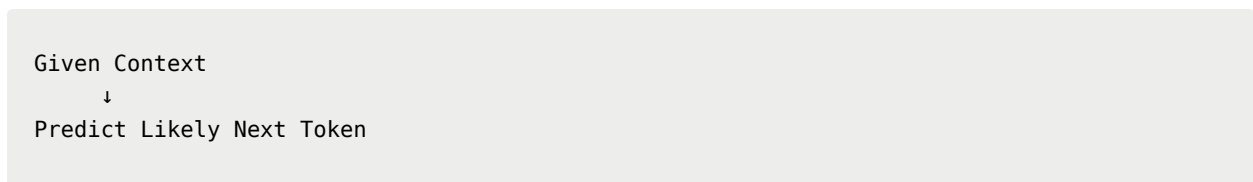
Streaming mainly improves the **perceived responsiveness** of the application.

It does not mean the underlying model suddenly generates the complete answer faster.

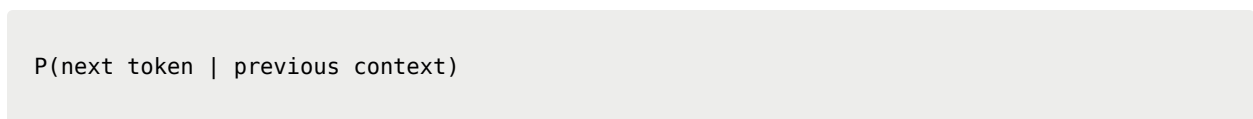
30. Hallucination

One of the most important limitations of LLMs is **hallucination**.

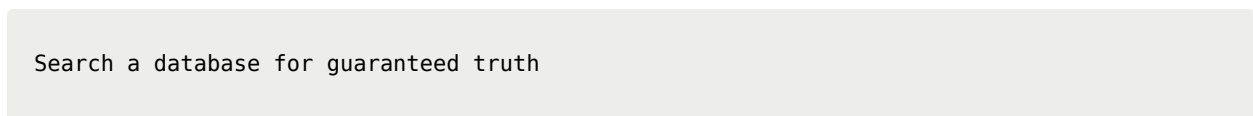
Start from the fundamental behaviour of a language model:



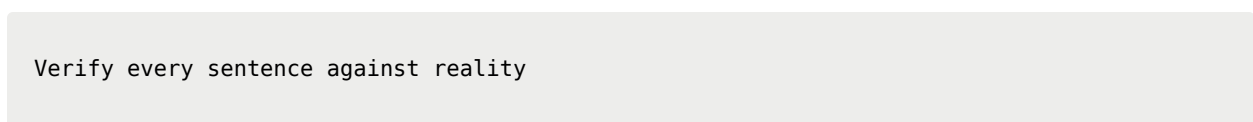
At a high level:



Notice what this mechanism is not automatically doing:



or:



or:

```
Return only confirmed facts
```

The model generates a plausible continuation based on its learned patterns and available context.

That distinction explains why hallucination is possible.

31. Simple Hallucination Example

Ask:

```
Who won the 2038 Cricket World Cup?
```

If 2038 has not happened yet, the correct response should recognise that there is no winner.

But a poorly grounded response could generate something like:

```
India defeated Australia in the final...
```

Why can this sound convincing?

Because sentences such as:

```
X defeated Y in the final by Z runs.
```

are perfectly normal patterns in cricket-related text.

The sentence can be linguistically excellent while describing an event that never happened.

32. Fluency and Truth Are Different

An LLM response can be:

Fluent	✓
Grammatically Correct	✓
Detailed	✓
Confident-Sounding	✓
Factually Correct	✗

This is an important misconception to remove.

People often assume:

Sounds Intelligent
=
Must Be Correct

That assumption is false.

Fluency and truth are different dimensions.

33. Why Does Hallucination Sound Confident?

A common question is:

If the model doesn't know, why doesn't it simply say "I don't know"?

Because the model is not necessarily experiencing certainty and uncertainty in the same way a human does.

It generates tokens.

Even phrases such as:

The answer is definitely...

are themselves generated text.

Therefore, confident wording should not automatically be treated as evidence that the underlying information is correct.

Confidence in language is not evidence of confidence in truth.

34. Common Situations That Can Lead to Hallucination

Hallucinations can occur when there is:

1. Missing Knowledge or Insufficient Information

The model does not have enough reliable information to answer properly.

2. Outdated Knowledge

The required information has changed since the model's available knowledge or context.

3. Ambiguous or Misleading Context

The input itself leads the model toward an incorrect interpretation.

35. Hallucination Is Not Usually Random Nonsense

An obvious incorrect answer such as:

```
Java was created by banana spaceship blue.
```

is not very dangerous because nobody is likely to believe it.

The more dangerous hallucination is something like:

```
Java was created in 1992 by James Gosling at IBM  
as part of Project Oak.
```

It looks believable.

Most of the sentence may even resemble real information, while one or more details are incorrect.

That is what makes hallucinations difficult.

Hallucination is often coherent fabrication, not obvious gibberish.

36. A Useful Hallucination Experiment

Ask the model about something intentionally fabricated but believable:

Explain Aditya Tandon's Law of Constant-Time Complexity Sorting, proposed by Aditya Tandon in 2018.

There is a false assumption inside the question.

A model that blindly accepts the premise may start inventing:

- a definition
- history
- examples
- mathematical explanations
- applications

A better-behaved model may challenge the premise and say that it cannot verify such a law.

Both outcomes create a useful lesson:

AI reliability is not something we should simply assume. It must be engineered and evaluated.

37. How GenAI Applications Reduce Hallucinations

A model alone may not have access to reliable, current or private information.

Applications can improve grounding using techniques such as:

```
RAG
Search
Tools
External APIs
Structured Outputs
Validation
Evaluations
Guardrails
```

These techniques can reduce hallucinations.

They do not guarantee that hallucination becomes exactly zero.

This is why production GenAI systems require more than simply calling an LLM.

38. The Bigger FDE Lesson

Building an AI application is not simply:

```
User
↓
LLM
↓
Response
```

A real application may look more like:

```
User
↓
Application Logic
↓
System Instructions
↓
Conversation History
```

↓
Context Management
↓
Memory / Retrieval / Tools
↓
LLM
↓
Validation
↓
Response

The LLM is only one component.

The application around it determines:

- what the model sees
- what it remembers
- what instructions it follows
- what external information it can access
- how responses are validated
- how reliable the final user experience becomes

Final Takeaways

1. An LLM call is not automatically a conversation.

The application creates and maintains the conversation.

2. Roles give messages different meanings.

User, assistant and system messages serve different purposes.

3. Context is temporary working information.

It is different from the model's learned parameters.

4. Conversation history and context window are not the same thing.

The application may store a large history while sending only a relevant subset to the model.

5. More context is not always better.

Relevant context is more valuable than simply maximum context.

6. Giving information in a prompt does not retrain the model.

The model is using that information from its current context.

7. Streaming improves the user experience.

It allows generated tokens to reach the user progressively.

8. Fluent answers are not guaranteed to be factual.

LLMs can produce convincing hallucinations.

9. Hallucination is an engineering problem.

RAG, tools, search, validation and guardrails can reduce it, but reliability still needs to be designed carefully.

The goal of a good GenAI engineer is not simply to connect an application to an LLM. The goal is to build the right system around the LLM.